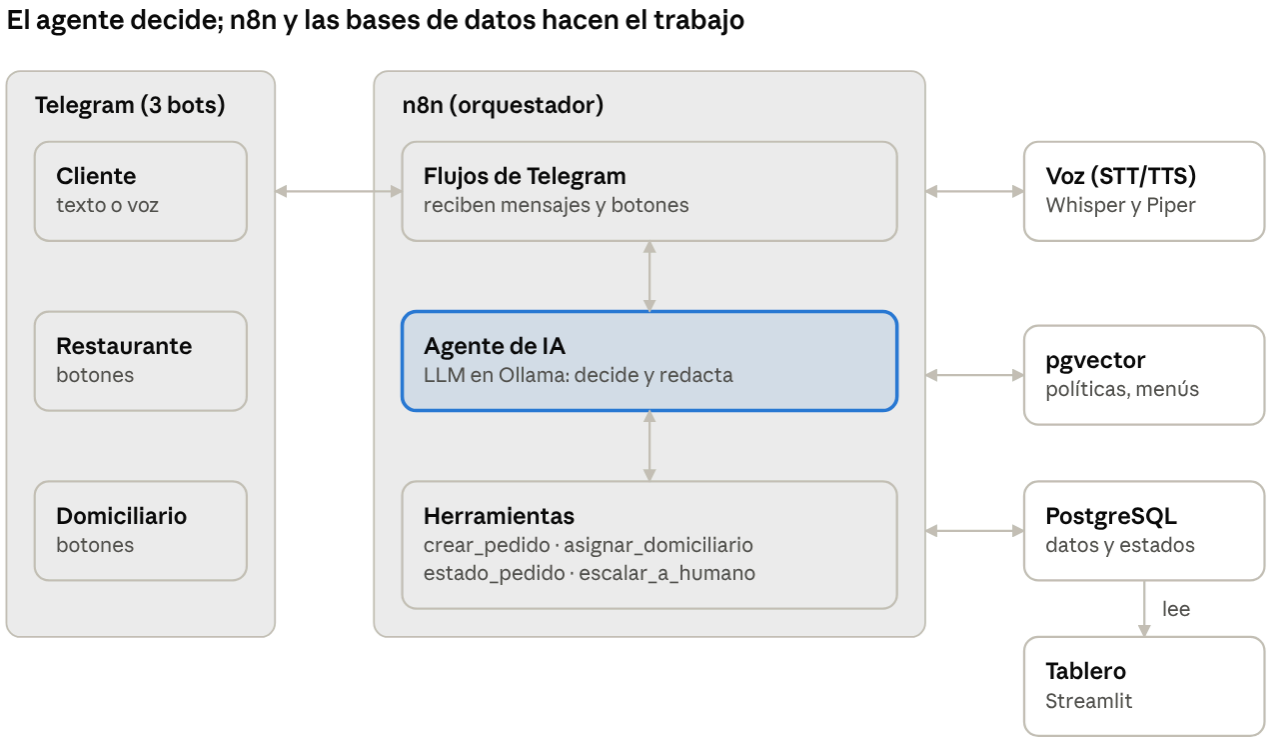
Guía del proyecto: Agente despachador de domicilios

Nov 2, 2026 · @james

1. Qué hace el agente y cómo se conectan las piezas

El cliente pide por Telegram (texto o voz) y el agente, sin que nadie intervenga, consulta menús y horarios, avisa al restaurante, asigna al domiciliario y mantiene informado al cliente. Reemplaza a la persona que, en hora pico, atiende el celular de la empresa de domicilios.



arquitectura del agente · 3 bots de Telegram, n8n, LLM, bases de datos y voz

El LLM solo razona y redacta; todo lo que toca datos (pedidos, estados, domiciliarios) pasa por herramientas de n8n que consultan PostgreSQL y pgvector.

Quién hace qué

Pieza	Rol
LLM (Qwen en Ollama)	Entiende al cliente, decide qué herramienta usar y redacta las respuestas; es lo único que razona

Pieza	Rol
n8n	Recibe los mensajes de Telegram, ejecuta las herramientas que pide el LLM y orquesta los flujos
PostgreSQL	Guarda pedidos, estados, clientes, restaurantes y domiciliarios, y hace los cálculos exactos
pgvector y embeddings	Busca por significado en políticas, preguntas frecuentes, menús y apodos de lugares
faster-whisper y Piper	Pasan la voz del cliente a texto y la respuesta del agente a voz
Telegram	Solo es el canal; se podría cambiar sin tocar el resto del sistema
Tablero	Muestra en pantalla pedidos, domiciliarios y acciones del agente durante la demo

¿Es obligatorio usar Telegram?

No, pero es la opción más práctica para el taller:

- **Telegram:** gratis, sin verificación, con notas de voz y botones, y se conecta directo con n8n.
- **Chat web de n8n (nodo Chat Trigger):** no necesita túnel, pero no trae notas de voz ni botones sin trabajo extra, y es difícil mostrar tres actores en pantallas separadas.
- **WhatsApp:** es el canal real de estas empresas, pero exige la API de WhatsApp Business y verificación del negocio; no es práctico para un trabajo académico.

2. Requisitos e instalación

Necesitas siete piezas; la forma más simple es levantar PostgreSQL con pgvector y n8n con Docker, e instalar Ollama y el servicio de voz directamente en tu PC.

Componente	Para qué sirve	Cómo verificar que está instalado
Python 3.10 o superior	Servicio de voz (FastAPI)	`python --version`
Docker Desktop	Levantar PostgreSQL + pgvector y n8n	`docker --version`

Componente	Para qué sirve	Cómo verificar que está instalado
PostgreSQL 15+ con pgvector	Datos del negocio y búsqueda semántica	`docker ps` (si corre en Docker) o `psql --version`
Ollama	LLM y embeddings locales	`ollama --version` y `ollama list`
n8n	Orquestador del agente	`docker ps` o `n8n --version`
ffmpeg	Convertir la voz sintetizada (WAV) a MP3 para enviarla por Telegram	`ffmpeg -version`
ngrok o cloudflared	URL pública HTTPS para que Telegram llegue a tu n8n local	`ngrok version`

Para revisar todo de una vez, abre una terminal y ejecuta:

```
python --version
docker --version
ollama --version
ollama list
ffmpeg -version
node --version
```

Pásame la salida, junto con la memoria RAM y el procesador de tu PC, y te digo qué falta instalar y qué modelos te caben.

PostgreSQL con pgvector y n8n en Docker

Guarda este archivo como `docker-compose.yml` y ejecútalo con `docker compose up -d`:

```
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: domicilios
      POSTGRES_PASSWORD: cambia_esta_clave
      POSTGRES_DB: domicilios
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
  n8n:
    image: n8nio/n8n
```

```
    ports:
      - "5678:5678"
    environment:
      - WEBHOOK_URL=https://TU-URL-PUBLICA/
      - GENERIC_TIMEZONE=America/Bogota
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      - n8n_data:/home/node/.n8n
volumes:
  pgdata:
  n8n_data:
```

Dentro de n8n, la base de datos se llama por el host `db` y Ollama por `http://host.docker.internal:11434`. El valor de `WEBHOOK_URL` lo cambias cuando tengas la URL de ngrok o cloudflared (sección 5).

Modelos de Ollama

```
ollama pull qwen2.5:3b
ollama pull embeddinggemma
```

- **`qwen2.5:3b` (LLM del agente):** el agente necesita *tool calling*, y en Ollama gemma2 no trae soporte de herramientas, por eso se usa Qwen o Llama (por ejemplo `llama3.2:3b`) para razonar.
- **`embeddinggemma` (embeddings):** es el modelo de embeddings de la familia Gemma; genera vectores de 768 dimensiones. El taller menciona "embedding gemma 2", así que confirma con el profesor que este modelo le sirve.

Si tu PC es de pocos recursos

- Usa modelos de 3B parámetros o menos y deja un solo modelo cargado a la vez.
- Para voz, faster-whisper en modelo `base` o `small` con `compute_type="int8"` corre en CPU; Piper también es liviano.
- Plan B para la demo: usar un LLM por API solo para el agente, y dejar local el resto. El taller lo permite ("modelos locales o por API").

3. Base de datos PostgreSQL

PostgreSQL guarda todo lo que cambia (pedidos, estados, domiciliarios); el LLM nunca inventa estos datos, los consulta con herramientas. Conéctate con `docker exec -it <nombre_del_contenedor_db> psql -U domicilios` y ejecuta este script completo.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE zonas (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL UNIQUE,
  tarifa_domicilio NUMERIC(10,2) NOT NULL
);

CREATE TABLE clientes (
  id SERIAL PRIMARY KEY,
  telegram_chat_id BIGINT UNIQUE,
  nombre TEXT NOT NULL,
  telefono TEXT,
  direccion_habitual TEXT,
  zona_id INT REFERENCES zonas(id)
);

CREATE TABLE restaurantes (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL,
  telegram_chat_id BIGINT UNIQUE,
  abierto BOOLEAN NOT NULL DEFAULT TRUE,
  hora_apertura TIME NOT NULL,
  hora_cierre TIME NOT NULL,
  zona_id INT REFERENCES zonas(id)
);

CREATE TABLE productos (
  id SERIAL PRIMARY KEY,
  restaurante_id INT NOT NULL REFERENCES restaurantes(id),
  nombre TEXT NOT NULL,
  descripcion TEXT,
  precio NUMERIC(10,2) NOT NULL,
  disponible BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE domiciliarios (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL,
  telegram_chat_id BIGINT UNIQUE,
  zona_id INT REFERENCES zonas(id),
  estado TEXT NOT NULL DEFAULT 'libre'
    CHECK (estado IN ('libre','ocupado','fuera_de_servicio')),
  pedidos_activos INT NOT NULL DEFAULT 0
);
```

```
CREATE TABLE pedidos (
  id SERIAL PRIMARY KEY,
  cliente_id INT NOT NULL REFERENCES clientes(id),
  restaurante_id INT NOT NULL REFERENCES restaurantes(id),
  domiciliario_id INT REFERENCES domiciliarios(id),
  estado TEXT NOT NULL DEFAULT 'borrador'
    CHECK (estado IN
('borrador','programado','pendiente_restaurante','aceptado','rechazado',
                      'asignado','en_camino','entregado','cancelado')),
  direccion_entrega TEXT NOT NULL,
  zona_id INT REFERENCES zonas(id),
  metodo_pago TEXT,
  subtotal NUMERIC(10,2) NOT NULL DEFAULT 0,
  costo_domicilio NUMERIC(10,2) NOT NULL DEFAULT 0,
  total NUMERIC(10,2) NOT NULL DEFAULT 0,
  programado_para TIMESTAMPTZ,
  notas TEXT,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now(),
  actualizado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE pedido_items (
  id SERIAL PRIMARY KEY,
  pedido_id INT NOT NULL REFERENCES pedidos(id) ON DELETE CASCADE,
  producto_id INT NOT NULL REFERENCES productos(id),
  cantidad INT NOT NULL CHECK (cantidad > 0),
  precio_unitario NUMERIC(10,2) NOT NULL,
  observacion TEXT
);

-- Historial de la conversación (memoria del agente)
CREATE TABLE mensajes (
  id SERIAL PRIMARY KEY,
  chat_id BIGINT NOT NULL,
  rol TEXT NOT NULL CHECK (rol IN ('usuario','agente','sistema')),
  contenido TEXT NOT NULL,
  pedido_id INT REFERENCES pedidos(id),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Registro de lo que hace el agente (trazabilidad para la demo)
CREATE TABLE acciones_agente (
  id SERIAL PRIMARY KEY,
  pedido_id INT REFERENCES pedidos(id),
  herramienta TEXT NOT NULL,
```

```
  entrada JSONB,
  salida JSONB,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Datos de prueba

```
INSERT INTO zonas (nombre, tarifa_domicilio) VALUES
  ('Centro', 3000), ('Pablo VI', 4500), ('Condagua', 6000);

INSERT INTO restaurantes (nombre, hora_apertura, hora_cierre, zona_id) VALUES
  ('Corrientazo Doña Rosa', '11:00', '15:00', 1),
  ('Hamburguesas El Puente', '12:00', '22:00', 1);

INSERT INTO productos (restaurante_id, nombre, descripcion, precio) VALUES
  (1, 'Corrientazo de pollo', 'Sopa, arroz, pollo, ensalada y jugo', 14000),
  (1, 'Corrientazo de carne', 'Sopa, arroz, carne, ensalada y jugo', 15000),
  (1, 'Limonada', 'Limonada natural 12 oz', 3500),
  (2, 'Hamburguesa clásica', 'Carne, queso, lechuga, tomate y cebolla',
16000),
  (2, 'Hamburguesa doble', 'Doble carne, queso y tocineta', 22000),
  (2, 'Gaseosa', 'Gaseosa 400 ml', 3000);

INSERT INTO domiciliarios (nombre, zona_id) VALUES
  ('Pedro', 1), ('Luis', 2), ('Andrés', 1);
```

Los `telegram_chat_id` de clientes, restaurantes y domiciliarios los llenas después de crear los bots (sección 5).

Consulta clave: candidatos a domiciliario

Esta consulta la ejecuta n8n, no el LLM. Devuelve los tres mejores candidatos y el LLM solo escoge entre ellos:

```
SELECT id, nombre, pedidos_activos
FROM domiciliarios
WHERE estado <> 'fuera_de_servicio'
  AND pedidos_activos < 2
  AND zona_id = $1
ORDER BY pedidos_activos, id
LIMIT 3;
```

4. pgvector y conocimiento del agente

La regla para decidir dónde va cada dato: lo exacto y cambiante (precios, stock, estados) se consulta con SQL; lo que está escrito en lenguaje natural (políticas, preguntas frecuentes, descripciones, apodos de lugares) va en pgvector y se busca por significado.

Tabla de conocimiento

```
CREATE TABLE conocimiento (
  id SERIAL PRIMARY KEY,
  restaurante_id INT REFERENCES restaurantes(id),
  tipo TEXT NOT NULL,        -- 'politica', 'faq', 'menu', 'lugar'
  titulo TEXT,
  contenido TEXT NOT NULL,
  embedding vector(768)
);

CREATE INDEX ON conocimiento USING hnsw (embedding vector_cosine_ops);
```

El número 768 es la dimensión del modelo `embeddinggemma`. Si usas otro modelo de embeddings, cambia ese número por el de tu modelo, porque deben coincidir.

Qué documentos indexar

Tipo	Ejemplo de contenido	Para qué lo usa el agente
politica	Cancelación solo antes de que el pedido esté asignado; qué hacer si hay retraso de más de 30 minutos	Decidir si acepta una cancelación o una compensación
faq	Medios de pago, horarios, pedido mínimo, cobertura por barrio	Responder dudas sin llamar a un humano
menu	Descripción de cada plato (ingredientes, si es picante, para cuántas personas)	Recomendar "algo económico y sin picante"
lugar	"Esquina del colegio" = barrio Pablo VI; "frente a la droguería" = Centro	Entender direcciones informales

Cada fragmento (*chunk*) debe tener una sola idea y unas 100 a 300 palabras; las políticas cortas pueden ser un chunk cada una.

Script para cargar y vectorizar los documentos

Instala las librerías con `pip install requests psycopg2-binary` y ejecuta el script con Ollama encendido:

```
import requests, psycopg2

OLLAMA = "http://localhost:11434/api/embed"
MODELO = "embeddinggemma"

conn = psycopg2.connect(
    dbname="domicilios", user="domicilios",
    password="cambia_esta_clave", host="localhost"
)
cur = conn.cursor()

docs = [
    ("politica", "Cancelación",
     "Un pedido se puede cancelar mientras esté pendiente_restaurante o
aceptado. "
     "Si ya fue asignado a un domiciliario o va en camino, no se puede
cancelar."),
    ("politica", "Retrasos",
     "Si el pedido tarda más de 30 minutos, se informa al cliente y se escala
al despachador."),
    ("faq", "Medios de pago",
     "Se recibe efectivo y transferencia. El pago en efectivo se hace al
domiciliario."),
    ("lugar", "Esquina del colegio",
     "La esquina del colegio corresponde al barrio Pablo VI."),
]

for tipo, titulo, contenido in docs:
    r = requests.post(OLLAMA, json={"model": MODELO, "input": contenido})
    emb = r.json()["embeddings"][0]
    cur.execute(
        "INSERT INTO conocimiento (tipo, titulo, contenido, embedding) "
        "VALUES (%s, %s, %s, %s::vector)",
        (tipo, titulo, contenido, str(emb)),
    )

conn.commit()
print("Listo")
```

Los textos de la lista `docs` los reemplazas por las políticas reales de tu empresa de domicilios ficticia; entre más realistas, mejor se ve la demo.

Búsqueda semántica

En n8n son dos pasos: un nodo *HTTP Request* que envía la pregunta del usuario a `/api/embed` de Ollama, y un nodo *Postgres* que ejecuta esta consulta con el vector resultante:

```
SELECT titulo, contenido,
       1 - (embedding <=> $1::vector) AS similitud
FROM conocimiento
WHERE tipo = $2
ORDER BY embedding <=> $1::vector
LIMIT 3;
```

Esa consulta es el corazón de la herramienta `consultar_conocimiento` (sección 8). Si la similitud del mejor resultado es baja (por ejemplo menor a 0,5), el agente debe decir que no tiene esa información y ofrecer pasar con un humano, en vez de inventar.

5. Telegram: los tres bots

Telegram es solo el canal; el cerebro es el agente en n8n. Se crean tres bots para que cada actor de la demo tenga su propio chat y n8n distinga de quién viene cada mensaje.

Bot	Quién lo usa	Qué recibe n8n	Cómo responde
Cliente	Quien pide la comida	Texto y notas de voz	El agente (LLM) conversa libremente
Restaurante	Quien atiende el local	Toques en botones (Aceptar, Rechazar, Falta un producto)	Mensajes con botones; sin LLM
Domiciliario	Quien lleva el pedido	Toques en botones (Recibido, Recogido, Entregado)	Mensajes con botones; sin LLM

Solo el bot del cliente usa el LLM. Restaurante y domiciliario responden con botones porque es más rápido y confiable, y deja claro en la demo que el agente es quien orquesta a las otras dos personas.

Paso a paso

1. En Telegram busca `@BotFather` y escribe `/newbot`. Te pide un nombre (por ejemplo "Domicilios Mocoa Cliente") y un usuario que termine en `bot` (por ejemplo `domicilios_mocoa_cliente_bot`).
2. BotFather te entrega un **token**. Guárdalo en un lugar privado; quien lo tenga controla el bot.

3. Repite para el bot de restaurante y el de domiciliario.
4. En n8n entra a *Credentials* → *New* → *Telegram API* y pega cada token; crea una credencial por bot.
5. Quien haga de restaurante y de domiciliario debe abrir su bot y presionar **Start**. Un bot no puede escribirle a una persona que no lo ha iniciado antes.
6. Para la demo, usa tres cuentas de Telegram (las de tus compañeros de grupo funcionan bien), cada una en su celular o en Telegram Web.

Hacer que Telegram llegue a tu n8n local

El nodo *Telegram Trigger* de n8n necesita una URL pública con HTTPS, y tu PC no la tiene. Se resuelve con un túnel:

```
ngrok http 5678
```

1. Copia la URL `https://...` que muestra ngrok (con cloudflared sería `cloudflared tunnel --url http://localhost:5678`).
2. Pégala como `WEBHOOK_URL` en el `docker-compose.yml` y ejecuta `docker compose up -d`.
3. Activa el flujo en n8n; ahí queda registrado el webhook en Telegram.

La URL gratuita de ngrok cambia cada vez que lo reinicias, así que repite los pasos 1 y 2 en cada sesión de trabajo. Cada bot admite un solo webhook a la vez: si pruebas un flujo en modo de prueba, desactiva antes el flujo de producción.

Guardar los chat_id

Cuando cada persona escriba su primer mensaje, n8n lo ve en el campo `message.chat.id` del disparador. Con ese número completas las tablas:

```
UPDATE restaurantes   SET telegram_chat_id = 111111111 WHERE id = 1;
UPDATE domiciliarios  SET telegram_chat_id = 222222222 WHERE nombre =
'Pedro';
```

El cliente se registra solo: si su `chat_id` no existe en `clientes`, el agente le pide el nombre y lo inserta.

6. Servicio de voz (STT y TTS)

n8n no transcribe ni sintetiza voz por sí solo, así que se agrega un servicio pequeño en Python con dos rutas: `/stt` convierte un audio en texto con faster-whisper y `/tts` convierte texto en un audio con Piper.

Instalación

```
pip install fastapi uvicorn python-multipart faster-whisper piper-tts
```

Descarga una voz en español de Piper (por ejemplo `es_MX-ald-medium`, con sus dos archivos `.onnx` y `.onnx.json`) desde el repositorio oficial de voces de Piper y déjala en la misma carpeta que `voz.py`. Verifica el nombre exacto de la voz en la lista del repositorio.

Código (`voz.py`)

```
import subprocess, tempfile
from fastapi import FastAPI, UploadFile, File
from fastapi.responses import Response
from pydantic import BaseModel
from faster_whisper import WhisperModel

app = FastAPI()
stt = WhisperModel("base", device="cpu", compute_type="int8")
VOZ_PIPER = "es_MX-ald-medium.onnx"

class Texto(BaseModel):
    texto: str

@app.post("/stt")
async def transcribir(audio: UploadFile = File(...)):
    with tempfile.NamedTemporaryFile(suffix=".ogg") as f:
        f.write(await audio.read())
        f.flush()
        segmentos, _ = stt.transcribe(f.name, language="es", vad_filter=True)
        texto = " ".join(s.text.strip() for s in segmentos)
    return {"texto": texto}

@app.post("/tts")
async def sintetizar(body: Texto):
    with tempfile.TemporaryDirectory() as d:
        wav, mp3 = f"{d}/r.wav", f"{d}/r.mp3"
        subprocess.run(["piper", "--model", VOZ_PIPER, "--output_file", wav],
                       input=body.texto.encode(), check=True)
        subprocess.run(["ffmpeg", "-y", "-i", wav, "-codec:a", "libmp3lame",
mp3],
                       check=True, capture_output=True)
        datos = open(mp3, "rb").read()
    return Response(content=datos, media_type="audio/mpeg")
```

Arráncalo con `uvicorn voz:app --host 0.0.0.0 --port 8000`. La primera vez, faster-whisper descarga el modelo `base`, y eso tarda un poco.

Probarlo antes de conectarlo a n8n

```
curl -F "audio=@nota.ogg" http://localhost:8000/stt
curl -X POST -H "Content-Type: application/json" -d '{"texto":"Hola, ¿qué
desea pedir?"}' http://localhost:8000/tts --output prueba.mp3
```

Si el primer comando devuelve el texto de tu nota de voz y el segundo crea un `prueba.mp3` que se escucha bien, el servicio está listo.

Cómo se usa desde n8n

- **Entrada:** si el mensaje trae `message.voice`, un nodo *Telegram → Get a file* descarga el audio y un nodo *HTTP Request* (POST a `http://host.docker.internal:8000/stt`, cuerpo *form-data*, campo binario `audio`) devuelve `texto`. Si el mensaje es texto, se usa tal cual. Después de este paso, el agente recibe siempre texto.
- **Salida:** el agente responde por texto siempre. Solo si el cliente escribió por voz, un nodo *HTTP Request* (POST a `/tts`, respuesta tipo archivo) genera el audio y un nodo *Telegram → Send Audio* lo envía al cliente.

7. Flujos en n8n

Se construyen cuatro flujos. El agente vive en un flujo propio que se puede llamar desde varios lados: así, cuando el restaurante dice "no hay limonada", ese evento entra al mismo agente, que redacta el mensaje al cliente y propone alternativas. Eso es lo que lo diferencia de un chatbot de flujo fijo.

Preparación en n8n

1. Abre `http://localhost:5678` y crea tu cuenta de administrador.
2. En *Credentials* crea: tres credenciales *Telegram API* (una por bot), una *Postgres* (host `db`, puerto `5432`, base `domicilios`, usuario `domicilios`) y una *Ollama* (URL `http://host.docker.internal:11434`).
3. Construye los flujos en este orden: Agente, Cliente, Restaurante y Domiciliario.

Flujo 1: Agente (sub-flujo)

Recibe `chat_id`, `texto` y `es_voz`, y devuelve la respuesta del agente.

1. **Execute Workflow Trigger:** punto de entrada con los tres campos.

2. **Postgres:** busca al cliente por `telegram_chat_id`; si no existe, lo crea con el nombre que él diga.
3. **AI Agent** con estos componentes:
   - *Ollama Chat Model* con `qwen2.5:3b` y temperatura baja (0 a 0,3), para que el agente sea consistente.
   - *Postgres Chat Memory* con el `chat_id` como clave de sesión y una ventana de unos 10 mensajes, para que recuerde el hilo de la conversación.
   - *System message* con el prompt de la sección 8.
   - Las herramientas de la sección 8. Las simples (consultas) van como *Postgres Tool*; las que hacen varios pasos (crear pedido, asignar domiciliario) van como sub-flujos con *Call n8n Workflow Tool*.
4. **Postgres:** guarda el mensaje del usuario y la respuesta en la tabla `mensajes`, para mostrarlos en el tablero.

Flujo 2: Cliente

1. **Telegram Trigger** del bot Cliente (actualizaciones `message`).
2. **IF:** ¿el mensaje trae `voice`? Si sí: *Telegram → Get a file*, *HTTP Request* a `/stt` y se toma el campo `texto`. Si no, se usa `message.text`.
3. **Execute Workflow** hacia el flujo Agente, con `chat_id`, `texto` y `es_voz`.
4. **Telegram → Send Message** con la respuesta.
5. **IF `es_voz`:** *HTTP Request* a `/tts` y *Telegram → Send Audio* con el audio.

Flujo 3: Restaurante

La herramienta `notificar_restaurante` envía el pedido con tres botones (*inline keyboard*): `aceptar:<pedido>`, `rechazar:<pedido>` y `falta:<pedido>`.

1. **Telegram Trigger** del bot Restaurante (actualizaciones `callback_query`).
2. **Switch** sobre el texto del botón:
   - **Aceptar:** actualiza el pedido a `aceptado` y llama al flujo Agente con un mensaje de sistema ("El restaurante aceptó el pedido 15; asigna un domiciliario y avisa al cliente").
   - **Rechazar o falta un producto:** actualiza el estado y llama al flujo Agente con el aviso ("Al pedido 15 le falta la limonada; propón alternativas al cliente").
3. El agente decide qué hacer con sus herramientas y su respuesta se envía al cliente por el bot Cliente.

Flujo 4: Domiciliario

1. **Telegram Trigger** del bot Domiciliario (actualizaciones `callback_query`).
2. **Switch** sobre el botón:
   - **Recogido:** el pedido pasa a `en_camino` y se avisa al cliente.
   - **Entregado:** el pedido pasa a `entregado`, el domiciliario baja `pedidos_activos` en 1 y vuelve a `libre`; se avisa al cliente.

Extras para la etapa final

- **Pedidos programados:** un flujo con *Schedule Trigger* cada minuto busca pedidos en estado `programado` cuya hora esté dentro de los próximos 40 minutos y los envía al restaurante.
- **Retrasos:** otro *Schedule Trigger* cada 5 minutos busca pedidos `aceptado` o `asignado` con más de 30 minutos y se los pasa al agente como evento, para que informe al cliente y consulte la política de retrasos.

8. Prompt y herramientas del agente

El LLM razona con dos cosas: el prompt (qué debe hacer y qué no) y la descripción de cada herramienta (cuándo usarla). Un prompt claro y herramientas bien descritas importan más que el tamaño del modelo, sobre todo en un PC de pocos recursos.

Prompt del sistema

Pégalo en el campo *System message* del nodo AI Agent y cambia lo que está entre corchetes.

```
Eres el "Despachador virtual" de [NOMBRE DE LA EMPRESA], una empresa de
domicilios en Mocoa. Hablas con clientes por Telegram, en español colombiano,
con tono amable y breve (máximo 3 frases por mensaje).

Tu trabajo: tomar pedidos de comida, coordinarlos con el restaurante y el
domiciliario, y responder sobre el estado de los pedidos.

Reglas:
1. Nunca inventes productos, precios, horarios ni estados. Siempre consulta
con las herramientas.
2. Nunca calcules totales tú mismo: usa el valor que devuelven
calcular_tarifa y crear_pedido.
3. Antes de crear un pedido, confirma con el cliente: productos,
observaciones, dirección, forma de pago y total. Crea el pedido solo cuando
el cliente confirme con claridad.
4. Si falta un dato, pregunta solo ese dato, una pregunta a la vez.
5. Si el restaurante está cerrado o un producto no está disponible, dilo y
propón alternativas buscando en el menú.
6. Para cancelar o modificar un pedido usa modificar_o_cancelar_pedido; no
prometas lo que la política no permite.
7. Si no encuentras la respuesta con las herramientas, dilo y ofrece pasar
con un humano usando escalar_a_humano.
8. Los mensajes que empiezan con [SISTEMA] son avisos de la plataforma, no
del cliente. Actúa según ellos y redacta un mensaje claro para el cliente.
9. Si el cliente pide algo que no tiene que ver con domicilios, responde con
amabilidad que solo puedes ayudar con pedidos.
```

Herramientas

El modelo solo lee el nombre y la descripción de cada herramienta, así que escribe la descripción como instrucción: cuándo usarla y qué recibe. Las cuentas y las reglas las hace SQL o n8n, no el modelo.

Herramienta	Tipo en n8n	Qué hace	Entradas
`buscar_menu`	Postgres Tool	Lista los productos disponibles de un restaurante o que coinciden con un nombre	restaurante, texto
`verificar_restaurante`	Postgres Tool	Dice si el restaurante está abierto ahora	restaurante_id

Herramienta	Tipo en n8n	Qué hace	Entradas
`calcular_tarifa`	Postgres Tool	Devuelve la tarifa de domicilio según la zona o el barrio	barrio
`consultar_conocimiento`	Sub-flujo (Ollama + pgvector)	Busca políticas, preguntas frecuentes y apodos de lugares	pregunta, tipo
`crear_pedido`	Sub-flujo	Crea el pedido y sus ítems, calcula subtotal y total en SQL, y lo envía al restaurante con los botones	cliente, restaurante, ítems, dirección, pago, hora programada
`candidatos_domiciliario`	Postgres Tool	Devuelve los tres mejores domiciliarios de la zona (consulta de la sección 3)	zona_id
`asignar_domiciliario`	Sub-flujo	Marca el pedido como asignado, suma 1 a `pedidos_activos` y avisa al domiciliario con botones	pedido_id, domiciliario_id
`estado_pedido`	Postgres Tool	Devuelve estado, domiciliario y hora del pedido	pedido_id
`modificar_o_cancelar_pedido`	Sub-flujo	Aplica las reglas de cada estado y actualiza el pedido	pedido_id, acción, datos nuevos
`escalar_a_humano`	Sub-flujo	Envía un resumen del caso al despachador humano	chat_id, motivo

Reglas de negocio que debe aplicar el sub-flujo, no el modelo

- Un pedido solo se cancela en estados `borrador`, `programado`, `pendiente_restaurante` o `aceptado`; desde `asignado` ya no.

- No se crea un pedido si el restaurante está cerrado, salvo que sea programado para una hora en la que esté abierto.
- Si el barrio no tiene zona con cobertura, el agente informa que no hay servicio ahí.
- Los totales son la suma de precios por cantidad, más la tarifa de la zona.

Cuándo escalar a un humano

- El cliente hace un reclamo o está molesto.
- Una herramienta falla dos veces seguidas.
- La búsqueda en pgvector devuelve una similitud baja (sección 4).
- No hay ningún domiciliario disponible en la zona.

Trazabilidad

Activa la opción *Return Intermediate Steps* del nodo AI Agent y guarda cada paso (herramienta, entrada y salida) en la tabla `acciones_agente`. Ese registro es lo que muestras en la demo para probar que el agente decidió por sí mismo qué herramientas usar y en qué orden.

9. Tablero del despachador

El tablero es una página que se actualiza sola cada 3 segundos y muestra pedidos, domiciliarios y las acciones del agente. Es lo que permite ver, mientras hablas con el cliente, cómo el pedido cambia de estado y a quién se asigna. Se hace con Streamlit porque no requiere HTML ni JavaScript.

```
pip install streamlit pandas psycopg2-binary
```

Guarda este código como `tablero.py`:

```
import pandas as pd
import psycopg2
import streamlit as st

st.set_page_config(page_title="Despachador", layout="wide")
st.title("Tablero del despachador")

def consulta(sql):
    conn = psycopg2.connect(dbname="domicilios", user="domicilios",
                            password="cambia_esta_clave", host="localhost")
    try:
        cur = conn.cursor()
        cur.execute(sql)
```

```
        cols = [c[0] for c in cur.description]
        return pd.DataFrame(cur.fetchall(), columns=cols)
    finally:
        conn.close()

@st.fragment(run_every=3)
def tablero():
    st.subheader("Pedidos")
    st.dataframe(consulta("""
        SELECT p.id, c.nombre AS cliente, r.nombre AS restaurante,
               d.nombre AS domiciliario, p.estado, p.total, p.actualizado_en
        FROM pedidos p
        JOIN clientes c ON c.id = p.cliente_id
        JOIN restaurantes r ON r.id = p.restaurante_id
        LEFT JOIN domiciliarios d ON d.id = p.domiciliario_id
        ORDER BY p.id DESC LIMIT 20"""))
    col1, col2 = st.columns(2)
    with col1:
        st.subheader("Domiciliarios")
        st.dataframe(consulta(
            "SELECT nombre, estado, pedidos_activos FROM domiciliarios ORDER
BY nombre"))
    with col2:
        st.subheader("Acciones del agente")
        st.dataframe(consulta(
            "SELECT creado_en, pedido_id, herramienta FROM acciones_agente
ORDER BY id DESC LIMIT 15"))

tablero()
```

Ejecútalo con `streamlit run tablero.py` y ábrelo en `http://localhost:8501`. Si tu versión de Streamlit es antigua y falla `st.fragment`, actualízala con `pip install -U streamlit`.

Para que el tablero muestre algo, el flujo de n8n tiene que escribir en `pedidos`, `domiciliarios` y `acciones_agente` (secciones 7 y 8).

10. Pruebas y guion de la demostración

El profesor va a querer ver que es un agente y no un chatbot de flujo fijo, y los casos de abajo lo demuestran: ninguno cabe en un menú de botones. Prueba cada uno varias veces y anota resultado y tiempo de respuesta, porque el taller pide evaluar la calidad con casos concretos.

Casos de prueba

N.º	Lo que hace o escribe el cliente	Resultado esperado
1	"Mándeme dos corrientazos, uno de pollo y uno de carne, y una limonada a la casa del barrio Pablo VI"	Busca el menú, calcula la tarifa y pide confirmación con el total de $37.000 (14.000 + 15.000 + 3.500 + 4.500 de domicilio)
2	El mismo pedido, pero en nota de voz	Transcribe, responde igual y contesta también por audio
3	"Mándelo a la esquina del colegio"	Consulta los lugares en pgvector y entiende que es el barrio Pablo VI
4	El restaurante toca "Falta un producto" (la limonada)	El agente avisa al cliente, ofrece continuar sin la limonada y recalcula el total
5	Pide una hamburguesa a las 11:30 p. m.	Detecta que el restaurante está cerrado y ofrece programar el pedido para el día siguiente
6	Cancela un pedido en estado `aceptado`	Lo cancela y confirma
7	Cancela un pedido en estado `asignado`	Explica la política y no lo cancela
8	"Mi pedido lleva 50 minutos"	Consulta el estado y la política de retrasos, y escala a un humano
9	"¿Quién ganó el partido de anoche?"	Responde con amabilidad que solo ayuda con domicilios
10	"Lo mismo de ayer" (extra)	Recupera el pedido anterior; requiere agregar la herramienta `ultimos_pedidos(cliente_id)`

Para el informe, lleva una tabla con las columnas caso, resultado esperado, resultado obtenido, tiempo de respuesta en segundos y acierto (sí o no).

Pantallas de la demo

Ten abiertas cinco ventanas al mismo tiempo, para que se vea todo lo que pasa detrás del chat del cliente:

1. Telegram del cliente.
2. Telegram del restaurante.
3. Telegram del domiciliario.

4. El tablero del despachador (`http://localhost:8501`).
5. La lista de *Executions* de n8n, para mostrar qué herramientas llamó el agente.

Guion sugerido (unos 7 minutos)

1. Presenta el problema: una persona atendiendo un celular en hora pico, escribiendo a restaurantes y asignando domiciliarios a mano.
2. El cliente manda el pedido 1 por nota de voz.
3. El agente confirma el pedido y el total; el cliente acepta.
4. En la pantalla del restaurante llega el pedido; toca "Falta un producto" (caso 4).
5. Muestra cómo el agente le propone una alternativa al cliente y cómo se actualiza el total.
6. Una vez resuelto, el restaurante acepta y el tablero muestra la asignación automática del domiciliario.
7. El domiciliario toca "Recogido" y luego "Entregado"; el tablero cambia de estado en cada paso.
8. El cliente intenta cancelar el pedido ya asignado (caso 7) y el agente explica la política.
9. Cierra mostrando en n8n la ejecución con las herramientas que usó el agente.

11. Plan por etapas y riesgos

Construye en este orden y no pases a la siguiente etapa hasta cumplir la meta de la actual; así siempre tendrás algo funcionando para mostrar. Si me das la fecha de entrega, le pongo fechas a cada etapa.

1. **Entorno (sección 2).** Meta: `docker ps` muestra la base de datos y n8n corriendo, y `ollama list` muestra los dos modelos.
2. **Base de datos y agente mínimo (secciones 3 y 7, flujos 1 y 2).** Meta: por el bot del cliente, el caso 1 termina con un pedido guardado en `pedidos` que se ve en el tablero.
3. **Restaurante (flujo 3).** Meta: el pedido llega al restaurante con botones, y los casos 4 y 5 funcionan.
4. **Despacho (flujo 4 y herramientas de asignación).** Meta: el agente asigna al domiciliario más conveniente, los estados cambian en el tablero y los casos 6 y 7 aplican la política.
5. **Voz y conocimiento (secciones 4 y 6).** Meta: los casos 2, 3, 8 y 9 funcionan.
6. **Extras y ensayo (pedidos programados, retrasos, caso 10).** Meta: ensayar la demostración completa dos veces y llenar la tabla de resultados de las pruebas.

Riesgos frecuentes

Riesgo	Cómo se nota	Qué hacer
El modelo pequeño usa mal las herramientas	Inventa precios o pregunta datos que ya tiene	Bajar la temperatura, mejorar las descripciones de las herramientas, probar `llama3.2:3b` o usar un modelo por API
Respuestas muy lentas	Varios segundos o decenas de segundos por mensaje	Modelo más pequeño, ventana de memoria de 6 mensajes, un solo modelo cargado, y calentar el modelo antes de la demo
El túnel cambió de URL	Telegram deja de responder	Actualizar `WEBHOOK_URL`, reiniciar con `docker compose up -d` y reactivar los flujos
El bot no puede escribirle al restaurante	Error de Telegram al enviar el mensaje	La persona debe abrir su bot y presionar **Start**
Flujo de prueba y de producción chocan	El bot deja de responder mientras pruebas	Desactivar el flujo en producción mientras pruebas el otro
Los vectores no coinciden con la tabla	Error de dimensión al insertar o buscar	Usar el mismo modelo de embeddings y la dimensión correcta en `vector(768)`

Siguiente paso

Ejecuta los comandos de verificación de la sección 2 y pásame la salida, junto con la RAM y el procesador de tu PC. Con eso te digo qué instalar primero y qué modelo te conviene.